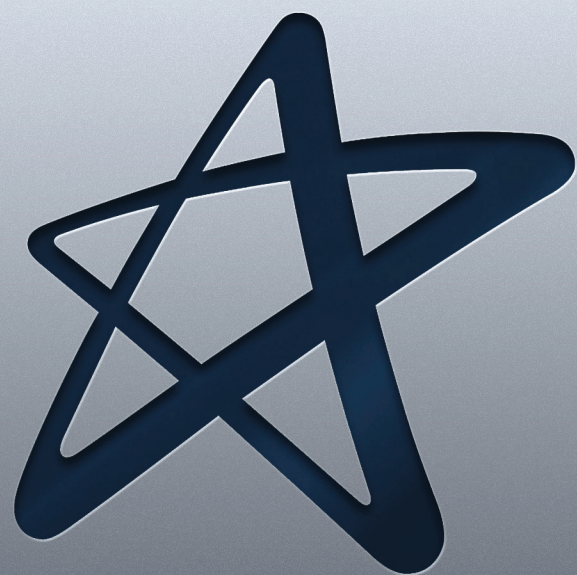


Compiladores e Interpretadores



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Fundamentos de Geração e Otimização de Código

Responsável pelo Conteúdo:

Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:

Prof.^a Dr.^a Luciene Oliveira da Costa Granadeiro

UNIDADE

Fundamentos de Geração e Otimização de Código



- Introdução;
- Geração de Código Intermediário;
- Otimização;
- Geração de Código.



OBJETIVO DE APRENDIZADO

- Apresentar todos os conceitos da geração e otimização de código.



Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

A compilação é composta por duas fases, fase de análise e síntese, correto? Na fase de análise, são realizadas análises no código-fonte. Uma vez feitas todas as análises necessárias, o próximo passo é gerar o código executável. A geração de código é realizada pela fase de síntese, também chamada de geração de código. Nesta unidade, vamos estudar todos os princípios da geração de código. Antes de estudarmos a geração de código, vale a pena relembrar alguns detalhes importantes. Observe a Figura 1.

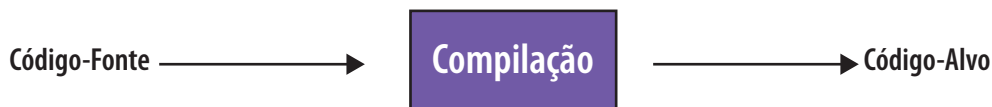


Figura 1 – Processo de Compilação

O processo de compilação toma como entrada o código-fonte e gera um código-alvo. O **código-alvo** é um código com instruções em nível de máquina. O objetivo do processo de compilação é realizar a tradução do código-fonte em código-alvo.

A fase de análise é responsável por analisar o código-fonte. Nessa fase, é verificado se as instruções do código-fonte estão corretas perante as regras gramaticais da linguagem. Podemos dizer, ainda, que é na fase de análise que é verificado se o que o programador escreveu está correto perante a linguagem.

As análises realizadas são três: análise léxica, sintática e semântica. Vale lembrar que, na fase léxica, são obtidos os *tokens* da linguagem, que são passados para o analisador sintático verificar a estrutura dos mesmos. A última análise a ser realizada nessa etapa é a análise semântica, que é responsável por verificar se as instruções estão semanticamente corretas perante a linguagem. As três análises são realizadas seguindo um fluxo. A Figura 2 ilustra o fluxo da realização de tais análises, ilustra o funcionamento de cada etapa também.

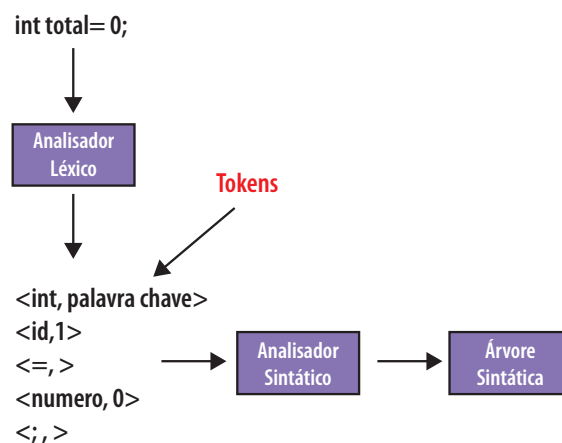


Figura 2 – Fluxo da realização das análises

Tendo essas informações claras em nossa mente, podemos começar o estudo sobre a segunda fase do processo de compilação. A geração de código é a fase mais

complexa do processo de compilação. Essa fase não depende só das características da linguagem, ela depende também de informações detalhadas da arquitetura do computador, de informações do sistema operacional pertencente ao computador-alvo e de outras informações. A fase de geração de código é dividida em 3 etapas:

- Geração de código intermediário;
- Otimização;
- Geração de código predita.

A geração de código intermediário deixa o código-fonte mais próximo da linguagem de máquina. A vantagem de se gerar código intermediário consiste em uma abstração na geração de código, isto é, com a geração de código intermediário, é possível gerar instruções que são independentes de arquitetura. A geração de código necessita de informações da arquitetura da máquina-alvo. Assim, se não houvesse a geração de código intermediário, o código-alvo ficaria “fixo” a uma única arquitetura. Agora, gerando código intermediário, é possível gerar instruções “genéricas” e, na etapa de geração de código, gerar as instruções em nível de máquina com informações para a arquitetura do computador-alvo. Há diversas técnicas para gerar código intermediário. Iremos estudar duas técnicas para gerar código intermediário e, posteriormente, gerar o código-alvo. As técnicas são: código de três endereços e P-Código. Ambas as técnicas são apresentadas com mais detalhes na Seção 2.

Já a etapa de **otimização** é responsável por otimizar o código-alvo, isto é, deixar o código alvo-mais rápido e mais enxuto. A vantagem também de se gerar código intermediário é a possibilidade de realizar otimizações no código-alvo antes de gerá-lo. Na Seção 3, são apresentadas algumas otimizações mais comuns realizadas no processo de compilação.

A última etapa é geração de código – feitas as otimizações necessárias, a próxima e última etapa é o código-alvo, código que terá instruções em nível de máquina. É fácil notar que o processo de compilação segue um fluxo lógico. Primeiro, são realizadas as análises necessárias, posteriormente, é realizada a geração de código intermediário, otimização e, por último, a geração de código predita. A Figura 3 ilustra todo o processo de compilação.

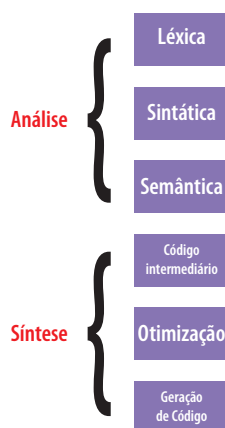


Figura 3 – Representação gráfica de todas as etapas do processo de compilação

Na próxima seção, estudaremos a etapa de geração de código intermediário.

Geração de Código Intermediário

A geração de código é fase complexa, por isso essa fase é decomposta em etapas, como mencionado anteriormente. A primeira etapa é a Geração de Código intermediário. Essa etapa pode usar **estruturas de dados intermediários** e frequentemente requerem alguma forma de código abstrato, denominado **código intermediário**.

No processo de geração de código, uma estrutura de dados que represente o programa-fonte durante a tradução é denominada **representação intermediária, ou IR**. Frequentemente, tal representação é realizada através de uma árvore de derivação.

Por que gerar um código intermediário em vez de gerar o código-alvo direto? Antes de entrarmos a fundo na geração de código intermediário, vamos entender por que gerar um código intermediário é importante.

Como **primeira razão para se gerar um código intermediário**, podemos citar que o código intermediário é particularmente útil quando o objetivo do compilador é produzir código extremamente eficiente, pois requer uma quantidade significativa de análise das propriedades do código-alvo, o que é facilitado pelo código intermediário.

Podemos citar como **outra vantagem** o reaproveitamento de código que facilita o transporte de um compilador para diversas plataformas de *hardware*; somente os módulos finais precisam ser refeitos a cada transporte.

O código intermediário deve possuir duas características importantes:

- ser fácil de produzir; e
- fácil de traduzir no programa-alvo.

Em uma forma bem resumida, a diferença básica entre o código intermediário e o código objeto final é que não são especificados detalhes da máquina-alvo, tais como quais registradores serão usados, quais endereços de memória serão referenciados etc.

Há duas formas populares de implementar um código intermediário, através das técnicas:

- Código de três endereços;
- P-código.

Começaremos o estudo pela técnica de código de três endereços, que é abordada na próxima seção.

Código de três endereços

Para a **geração de código para operações aritméticas** e também para a **geração de código para atribuições**, podemos realizar uma representação intermediária e, posteriormente, realizar a geração de código. Uma das formas mais comuns

e simples de implementar um código intermediário é através da técnica de **código de três endereços** (LOUDEN, 2004).

Na técnica de código de três endereços, a instrução mais básica é projetada para representar a avaliação de **expressões aritméticas**, e tem a seguinte forma geral:

$$x = y \text{ op } z$$

Nessa instrução, é aplicada uma operação *op* entre os valores *y* e *z* e a atribuição desse valor como novo valor de *x*. A operação *op* pode ser um operador aritmético como + ou –, ou algum outro operador que possa operar sobre os valores de *y* e *z*.

Uma curiosidade interessante sobre o código de três endereços é sobre o nome dessa técnica, o nome “código de três endereços” advém dessa forma de instrução, pois em geral cada um dos nomes *x*, *y* e *z* representa um endereço na memória.

Existem várias formas de implementar o código intermediário, todavia, todas as formas resultam em uma forma de linearizar a **árvore sintática**, ou seja, uma representação da **árvore de forma sequencial** (LOUDEN, 2004).

Para visualizar como a sequência de código de três endereços pode representar a computação de uma expressão, considere a expressão aritmética: $2 * a + (b - 3)$. Para essa expressão, a árvore sintática dela seria:

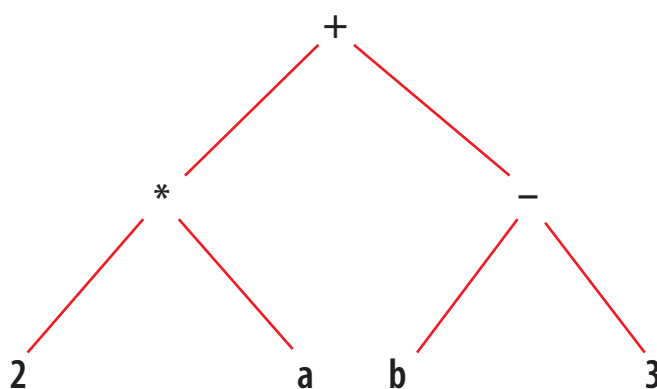


Figura 4 – Árvore sintática para $2 * a + (b - 3)$

Fonte: Adaptado de LOUDEN, 2004

Para essa expressão, o código de três endereços correspondente é:

$$t1 = 2 * a$$

$$t2 = b - 3$$

$$t3 = t1 + t2$$

Na técnica de código de três endereços, é necessário que o compilador gere nomes para os temporários, que denominamos nesse exemplo como: **t1**, **t2** e **t3**. Podemos observar que esses temporários correspondem aos nós interiores da árvore sintática e representam seus valores computados, com o temporário final (**t3**, nesse exemplo) representando o valor da raiz. O código de três endereços dado anteriormente representa uma linearização da esquerda para a direita da árvore

sintática, pois o código que corresponde à avaliação da subárvore à esquerda da raiz é listada primeiro (LOUDEN, 2004).

É interessante observar também que pode ocorrer outra ordem de instruções para esse código de três endereços:

O código de três endereços

$t1 = 2 * a$

$t2 = b - 3$

$t3 = t1 + t2$



O código de três endereços

$t1 = b - 3$

$t2 = 2 * a$

$3 = t1 + t2$

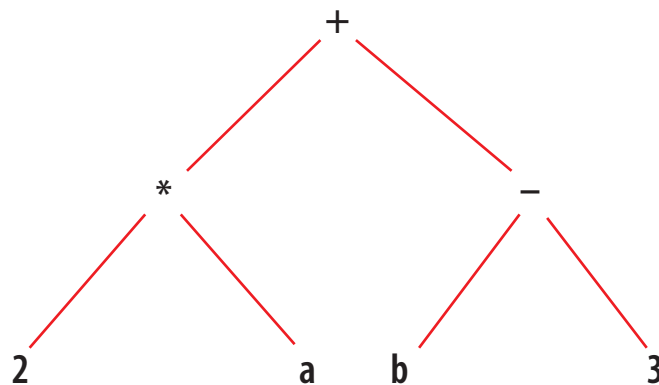


Figura 5 – Árvore sintática para $2*a + (b - 3)$ – Código de três endereços

Fonte: Adaptado de LOUDEN, 2004

A ordem de computação das instruções para código de três endereços pode ser;

$t1 = b - 3$

$t2 = 2 * a$

$t3 = t1 + t2$

Uma variante da técnica de código de três endereços é a técnica de **código de dois endereços**. Nessa técnica, pode ser realizada uma simples atribuição, tal como:

$t2 = t1$

Tipicamente, a etapa de código intermediário gera como saída uma representação intermediária “mais próxima” ao código-alvo. Podemos falar, ainda, que a etapa de código intermediário gera diferentes níveis de representação. Uma **representação de alto nível** pode ser dada através das **árvores de análise sintática**. Essa representação mostra a estrutura hierárquica natural das instruções e também permite realizar tarefas como checagem estática de tipos. Para uma **representação mais baixo nível**, podemos citar a técnica de código de três endereços. Essa representação é boa para tarefas dependentes de máquinas, geração de código, alocação de registradores, seleção de instruções e otimização (LOUDEN, 2004).

P-código

Outra forma de implementar um código intermediário é através da técnica P-código. Essa técnica foi projetada como código de máquina hipotética baseada em pilhas, denominada P-máquina. A ideia dessa técnica é facilitar a transportabilidade dos compiladores. Essa técnica é mais próxima ao código de máquina. Vamos apresentar um exemplo bem simples. Considere a mesma instrução apresentada anteriormente, $2^*a + (b - 3)$; usando a técnica do P-código, teríamos:

```
ldc 2      ; carrega a constante 2
lod a      ; carrega o valor da variável a
mpi        ; multiplicação de inteiros
lod b      ; carrega o valor da variável b
ldc 3      ; carrega a constante 3
sbi        ; subtração de inteiros
adi        ; adição de inteiros
```

Como é possível observar, essa técnica gera instruções mais próximas de nível de máquina (LOUDEN, 2004).

Diferença Código de três endereços e P-código

Há bastantes diferenças entre a técnica de código de três endereços e o P-código. Na técnica de código de três endereços, temos:

- Código de três endereços possui menos instruções;
- É autossuficiente, isto é, não precisa de uma pilha para representar o processamento.

Já para o P-código, temos:

- O P-código é mais próximo do código de máquina;
- O P-código utiliza uma pilha implícita;
- No P-código, as instruções exigem menos endereços.

A geração de código predita pode ser realizada com a técnica de três endereços ou com a técnica P-Código. Por isso, na Seção 4 “Geração de Código”, são apresentados mais detalhes sobre a técnica de três endereços e a técnica de P-Código voltada para a geração e código.

Na próxima seção, iremos estudar a etapa de otimização do compilador.

Otimização

A fase de geração de código envolve também a etapa de **otimização**, isto é, melhorar a velocidade e ou o tamanho do código gerado. Habitualmente, a otimização se dá através de coleta de informações adicionais sobre o programa fonte e o ajuste do código gerado para fazer uso das características específicas da máquina-alvo (máquina onde o executável irá rodar), tais como registradores, modos de endereçamento, níveis de memória, entre outras informações.

Uma otimização comum que o compilado realiza é não gerar instruções de máquinas de comandos que não fazem sentidos no código fonte, ou, ainda, não gerar instrução para variáveis ou métodos que não são utilizados. Por exemplo, considere o código a seguir:

```
public class Teste {  
  
    public static void main(String[] args) {  
        int aux = 0;  
  
        for(int i = 0; i < 10; i++){  
            System.out.println("O valor de i é " + i);  
        }  
    }  
}
```

Figura 6 – Código exemplo para otimização – I

Nesse código, a variável `aux` é declarada, todavia, nunca é utilizada. Assim, para otimizar o código-alvo (aquele que será gerado após a compilação), o compilador não gera a instrução para declarar `aux`. Assim, o código-algo ficará mais rápido, pois não conterá instruções para o comando “`int x = 0`”.

Agora, vamos supor outra situação, observe o código a seguir:

```
public class Teste {  
  
    public static void main(String[] args) {  
        int aux = 0;  
        for(int i = 0; i < 10; i++){  
            System.out.println("O valor do i é " + i);  
            aux++;  
        }  
    }  
}
```

Figura 7 – Código exemplo para otimização – II

Agora, nesse código, o aux é incrementado dentro do laço for. Todavia, o valor contido em aux nunca é usado. Para otimizar o código, também não serão geradas instruções para o aux++.

Vamos supor outra situação, observe o código a seguir:

```
public class Teste {  
  
    public static void main(String[] args) {  
        int aux = 0;  
        for(int i = 0; i < 10; i++){  
            System.out.println("O valor do i é " + i);  
            aux++;  
        }  
  
        System.out.println("O valor do aux é " + aux);  
    }  
}
```

Figura 8 – Código exemplo para otimização – III

Nesse código, o valor do aux é utilizado no “System.out.println”. Dessa forma, serão geradas instruções para o aux = 0, para o aux++ e para “System.out.println”.

A etapa de otimização tenta realizar otimizações semelhantes a essas para que o código-alvo fique mais rápido ou mais enxuto. A próxima e última etapa realizada na fase de síntese é a geração de código predita. Nesta etapa realmente são geradas as instruções em nível de máquina presentes no código-fonte. Na geração de código predita, há bastantes assuntos para serem estudados. Assim, o estudo da geração de código predita se inicia nesta unidade e se estende até a próxima unidade.

Geração de Código

Normalmente, a fase de geração de código sempre segue o fluxo das etapas – primeiro, a geração de código intermediário, depois, a otimização, depois, a geração de código predita. Habitualmente, a geração de código a partir de código intermediário requer as técnicas:

- expansão de macros; e
- simulação estática.

A técnica **expansão de macros** requer a substituição de cada tipo de instrução de código intermediário por uma sequência de instruções de código-alvo. Assim, é necessário que o compilador acompanhe as decisões sobre localização e particularidades de código em estruturas de dados separadas e que os procedimentos de macros variem a sequência de código conforme solicitado pelos tipos específicos de dados que ocorrem na instrução de código intermediário.

Já a **simulação estática** requer uma simulação direta dos efeitos do código intermediário e a geração de código-alvo correspondente a esses efeitos. Essa técnica também requer estruturas de dados adicionais e pode variar de formas muito simples de acompanhamento usadas em combinação com a expansão de macros até a altamente sofisticada **interpretação abstrata** (LOUDEN, 2004).

Para entender melhor essas técnicas, vamos considerar o código de três endereços e o P-código. Considere a seguinte expressão: $(x = x + 3) + 4$. Em **P-código** essa expressão seria:

```
lda x
lod x
ldc 3
adi
stn
ldc 4
adi
```

Já em três endereços:

```
t1 = x + 3
x = t1
t2 = t1 + 4
```

Vamos considerar uma tradução dos três endereços para o P-Código. No P-código, é necessária uma estrutura de dados em pilha. Assim, uma pilha para o P-código inicialmente poderia ser:

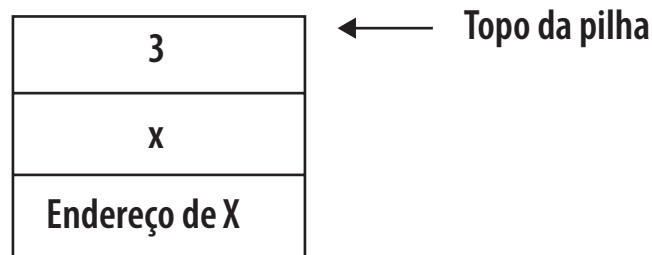


Figura 9 – Pilha de P-Código
Fonte: Adaptado de LOUDEN, 2004

Agora, é processada a operação **adi**; a instrução de três endereços:

```
t1 = x + 3
```

é gerada e, conseqüentemente, a pilha é alterada para:

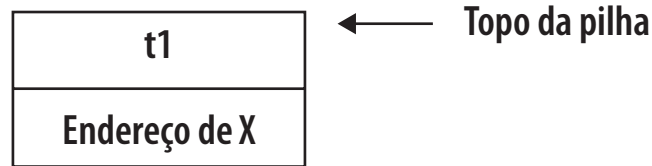


Figura 10 – Pilha de P-Código
Fonte: Adaptado de (LOUDEN, 2004)

A instrução stn gera a instrução de três endereços:

$$x = t1$$

Assim, a pilha é alterada para:



Figura 11 – Pilha para $x = t1$
Fonte: Adaptado de LOUDEN, 2004

A instrução seguinte coloca a constante 4 na pilha:



Figura 12 – Pilha para t1 4
Fonte: Adaptado de LOUDEN, 2004

Agora, a instrução adi gera a instrução de três endereços:

$$t2 = t1 + 4$$

Assim, a pilha é alterada para:



Figura 13 – Pilha para t1
Fonte: Adaptado de (LOUDES, 2004)

Finalmente, a simulação estática e a tradução são completadas.

Agora, vamos considerar o caso da tradução do código de três endereços para o P-Código. Podemos fazer isso através da expansão simples de macros. Logo, uma instrução de três endereços, como:

$$a = b + c$$

pode ser traduzida para a sequência em P-Código:

lda a

lod b ; or ldc b if b is a const

```
lod c      ; or ldc c if c is a const  
adi  
sto
```

Logo, isso resultaria na tradução a seguir do código de três endereços em P-Código:

```
lda t1  
lod x  
ldc 3  
adi  
sto  
lda x  
lod t1  
sto  
lda t2  
lod t1  
ldc 4  
adi  
sto
```

Vale ressaltar que lod significa carregar, adi adição sto e stn para armazenamento e lda para carregar o endereço. P-código é mais complexo do que o código de três endereços. Portanto, é normal que o P-Código gere mais instruções que o código de três endereços.



Em Síntese

A fase de síntese é a última fase realizada pelo processo de compilação. Nela, são geradas as instruções de máquina que irão compor o código-alvo. A geração de código é complexa, assim, essa fase é dividida em 3 etapas, geração de código intermediário, otimização e geração de código predita. A geração de código intermediário objetiva deixar o código-fonte mais próximo da linguagem de máquina. Já a etapa de **otimização** é responsável por otimizar o código-alvo, isto é, deixar o código-alvo mais rápido e mais enxuto. A última etapa é geração de código – feitas as otimizações necessárias, a próxima e última etapa é o código-alvo, código que terá instruções em nível de máquina.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Construindo Compiladores

COOPER, K. D.; TORCZON, L. **Construindo Compiladores**. 2. ed. Rio de Janeiro: Elsevier, 2014.

Optimizing compilers for modern architectures: a dependence-based approach

KENNEDY, K.; ALLEN, J. R. **Optimizing compilers for modern architectures: a dependence-based approach**. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2002.

Leitura

Basics of Compiler Design

MOGENSEN, T. Æ. **Basics of Compiler Design**. 2010. (e-book)

<http://bit.ly/2QONCVs>

Compiler Construction

WAITE, W. M.; GOOS, G. **Compiler Construction**. 1996. (e-book)

<http://bit.ly/2PqDxxy>

Referências

AHO, A. V. **Compiladores: Princípios, Técnicas e Ferramentas**. 2. ed. Pearson, 2007. (*e-book*)

APPEL, A. W. **Modern Compiler Implementation In Java**. 2. ed. New York: Cambridge University Press, 2002.

BERGMANN, S. D. **Compiler Design: Theory, Tools, and Examples**, 2016. <<http://www.cs.cmu.edu/~aplatzer/course/Compilers/waitegoos.pdf>>. (*e-book*)

LOUDEN, K. C. **Compiladores: Princípios e Práticas**. São Paulo: Pioneira Thomson Learning, 2004.

PRICE, A. M. A. **Implementação de Linguagens de Programação: Compiladores**. 2. ed. Porto Alegre: Sagra Luzzatto, 2001.



Cruzeiro do Sul
Educatonal